When the Semantic Layer Is Wrong: Do LLM Data Agents Follow or Flag Faulty Business Definitions?

Poorva Barve
poorvabarve@gmail.com

***Abstract*—Semantic layers improve text-to-SQL accuracy when their business definitions are correct. We study 24 controlled faults in an Apache Ossie model over TPC-DS, covering formulas, filters, synonyms, fiscal calendars, join keys and value mappings. Each fault has a coherent form, whose documentation agrees with it, and a contradictory form, whose documentation retains the reference definition. In 1,512 single-turn, tool-free trials with three models, a correct layer raised accuracy from 60–71% to 100% on this benchmark. Coherent faults were followed in 46–73% of trials and reduced pooled accuracy relative to no layer by 26 percentage points (95% CI, 9–43). Contradictory faults were located in 92–100% of trials but followed in 4–33%. Some responses raised a concern about the faulted definition and still used it. A generic audit instruction did not reliably reduce following and raised concerns on the correct layer by 17 percentage points. Plausible business conventions may require an independent policy reference to verify. Author annotations of 417 sampled concerns gave agreement with the detection rule of Cohen's $\kappa = 0.89$ for layer concerns and 0.90 for located faults.**

***Index Terms*—semantic layer, text-to-SQL, LLM agents, data governance, robustness, silent failure**

I. Introduction

LLM data agents increasingly answer business questions through a semantic layer instead of raw tables. Examples include dbt's semantic layer, Snowflake Cortex Analyst semantic models, Cube, and Apache Ossie (incubating) [1], a vendor-neutral YAML format formerly called the Open Semantic Interchange. A semantic layer defines what "revenue", "fiscal year" or "preferred customer" mean, how tables join, and which words are synonyms.

Several studies report accuracy gains from providing correct semantic context [2]–[5]. Layers, however, are maintained by hand, and they drift: a metric is redefined, a filter is added for one report, a synonym is attached to the wrong metric, a join key is mistyped. A layer is dangerous when it is wrong *because* it is authoritative. An agent that follows a wrong definition writes a query that runs and returns a plausible number, with no sign that anything is amiss.

We ask what agents do when the layer is wrong:

- **RQ1:** How much does a correct layer help?
- **RQ2:** Is a wrong element worse than a missing one, or than no layer at all?
- **RQ3:** When the layer is wrong, do agents follow it silently? Does it matter whether the fault can be seen by reading the layer?
- **RQ4:** Does asking agents to audit the layer help, and what does it cost?

To answer them we built a benchmark of 24 business questions over TPC-DS [6], each anchored on one element of an Ossie semantic model, and injected a controlled fault of one of six types into that element. Each fault comes in a coherent form, whose documentation agrees with the fault, and a contradictory form, in which only the definition changes. Three models were evaluated across 13 conditions, with five conditions using a 12-question subset: 1,512 trials in all. Every extracted SQL query was executed on Snowflake and its result compared with both the correct answer and the answer the faulty definition produces. Agents also reported structured concerns, which lets us measure *flagging* separately from *following*.

Our contributions are:

- A fault-injection benchmark for semantic layers: six typed fault operators in coherent and contradictory forms, a gate that guarantees each fault changes the answer on the data, two independently written gold sets, and deterministic grading against Snowflake.
- Evidence from protocol-specified comparisons that coherent faults are followed far more often than contradictory ones ($+42$ percentage points) and that a wrong element is worse than a missing one ($+33$ percentage points); a coherent wrong element is also worse than no layer at all.
- The finding that flagging and following are not exclusive: agents often state that a definition looks wrong and then use it.
- Evidence that a generic audit instruction raises concerns on a correct layer without reliably reducing following, with implications for how products built on semantic layers present agent concerns.

II. Related Work

Semantic layers for LLM analytics. Rumiantsau and Fokeev [2] compare a schema-only prompt with a schema plus a semantic layer in a paired benchmark over three models; they define "silent hallucination" but do not measure it. QUVI [3] routes an agent through a hand-curated layer, GROUND [4] reduces hallucination with governed definitions, and Lee [5] treats the layer as the arbiter of business truth while noting that an error in a metric definition can escape both of its checks. All of these studies assume the layer is correct.

Preprint, November 2026. Not peer reviewed.

Wrong evidence in text-to-SQL. The evidence field of BIRD [7] is in effect a small semantic layer. SEED [8] finds incorrect evidence in 105 of 1,534 BIRD development pairs (6.84%) and shows that correcting it raises accuracy, and Jin et al. [9] correct the external knowledge in 17 of the 100 examples they re-annotate. These studies analyze naturally occurring annotation errors, and SEED categorizes their types. Our experiment instead injects controlled changes into a full semantic model and scores result matching and reported concerns separately.

Robustness benchmarks. Dr.Spider [10], EvoSchema [11] and SAGE [12] perturb the question or the schema, either preserving meaning or changing the database itself; none plants a wrong definition over unchanged data. R-NL2SQL [13] formalises auxiliary documentation as an input to the task but does not perturb it.

Silent failure. DiscRun-Bench [14] scores data correctness and caveats separately for agents doing data preparation. Mismatches between documentation and data format almost never surfaced: they were missed in 18 of 19 and 18 of 18 runs.

Knowledge conflicts and documentation errors. LLMs readily follow coherent, convincing context that conflicts with what they know, and they often notice a conflict without locating it [15]. In code, incorrect docstrings hurt models' code understanding more than missing ones [16], the closest precedent for "wrong is worse than missing".

Our study complements this work by measuring both following and flagging after controlled semantic-layer changes, and by contrasting coherent with contradictory documentation. This is a targeted comparison with related work; it does not establish that the underlying knowledge-conflict mechanism is new.

III. Benchmark and Method

The protocol, hypotheses, outcome definitions and analysis plan were fixed before the main run. Changes made after the pilot are documented in a post-pilot amendment (Section III-I).

A. Data and Execution

TPC-DS at scale factor 1 was generated deterministically with DuckDB's `dsdgen` [17] and loaded into Snowflake; all 24 tables were verified column by column. Agents write Snowflake SQL. Each query is first checked with `sqlglot` [18] to be a single `SELECT`, then executed in an isolated container by a read-only key-pair user with a 120 s statement limit.

B. Semantic Layer

The base layer is the Ossie TPC-DS example at revision `2fd19c7a` [1], with logged edits: sources were retargeted; three metric descriptions were made explicit about their formulas; fields, metrics and synonyms were added for the fault operators; two conflicting synonyms were removed; and a fan-out error in the example's `store_productivity` metric was fixed, which divided by a sum of employee counts taken over joined sales rows. All 75 layer files used in the study validate against Ossie's JSON schema.

TABLE I
Fault operators, with one example each

Operator	Example fault
Formula drift	`total_sales` sums `ss_net_paid` instead of `ss_ext_sales_price`
Hidden filter	`distinct_customers` counts only customers on profitable lines (`ss_net_profit > 0`)
Synonym	"department" is mapped to `i_class` instead of `i_category`
Fiscal definition	`fiscal_year` and `fiscal_quarter` start the fiscal year in July instead of February
Relationship key	sales join customers on `ss_addr_sk = c_customer_sk`
Value mapping	`store_size_class` calls a store large from 240 employees instead of 270

C. Questions and Faults

There are 24 business questions, four for each of six fault operators (Table I). Each question is anchored on one business definition; its fault changes the corresponding element or, for fiscal definitions, the two related fields. The fault has two forms:

- **F1, coherent:** the element's description is edited to agree with the fault, so the layer is internally consistent.
- **F2, contradictory:** only the expression, mapping or join changes, so the definition disagrees with its own documentation.

Every (question, fault) pair passes a discriminability gate: on the data, the faulty definition gives a different answer from the reference definition. Here, "faulty means a departure from the benchmark reference. A coherent change to a business convention, such as the fiscal-year start, may be impossible to distinguish from a legitimate policy change using the supplied context alone. These trials measure reliance on the supplied definition; they do not assume the agent can recover an undisclosed policy.

Gold queries were written twice. The second set was written by a separate model that saw only the questions, the layer and the DDL. The two sets agree on all 24 questions, and DuckDB and Snowflake return identical results.

D. Arms

Table II lists the thirteen arms. The audit instruction reads: "Before answering, check the semantic model against the table definitions and the question, and report any problem you find in 'concerns'." The named-suspect instruction instead asks the agent to check "the semantic model's definition of" the anchored element, or of the synonym for synonym faults.

E. Agents

The models were Claude Opus 5.5 (high effort), GPT-6.1 Sol (high reasoning) and Claude Haiku 5.5 (medium effort), with two repetitions per question and arm: 1,512 trials in total. Each trial is a single turn with no tools, so the agent cannot run queries. The system prompt is identical across arms. It says that the organization maintains a semantic model, and it tells the agent neither to trust the model nor to verify it. The agent

TABLE II
ARMS. N, C1A AND C1B USE A 12-QUESTION SUBSET

Arm	Shown to the agent	Questions
C0	Table definitions (DDL) only	24
C1	DDL and the correct layer	24
E	Correct layer with the anchored element removed	24
F1, F2	Faulty layer (coherent, contradictory)	24
C1+A, F1+A, F2+A	As C1, F1, F2, plus the generic audit instruction	24
C1+N, F1+N, F2+N	As C1, F1, F2, plus an instruction naming the anchored element as suspect	12
C1a, C1b	Correct layer without metrics; without descriptions and `ai_context`	12

TABLE III
DETECTION RULE AGAINST THE AUTHOR'S LABELS

Label	n	Agree	κ [95% CI]	Prec.	Recall
D1 (about the layer)	417	0.945	0.89 [0.85, 0.93]	0.995	0.898
D2 (names the fault)	219	0.954	0.90 [0.84, 0.96]	0.962	0.962
D2 without Q13–Q16	185	0.951	0.90 [0.83, 0.96]	0.964	0.955

answers with one JSON object holding `sql`, `assumptions`, a list of structured `concerns` (each with a target of *layer*, *question*, *data* or *other*, an object identifier and a short text) and a `confidence`.

F. Outcomes

All outcomes are computed by deterministic code frozen before dispatch. Each answer is classed as *correct* (its result equals the gold result, including equivalent forms that the layer's descriptions license), *followed* (its result equals the faulty definition's result), *other wrong*, or *no usable answer*. **D1** is a layer-targeted concern that names something in the layer shown. **D2** is a D1 concern that names the faulted element, an identifier the fault changed, or the moved synonym. A **silent failure** is a followed answer with no D1 concern. On the correct layer, D1 measures concerns raised about a layer with no planted fault.

G. Human Check of the Detection Rule

The author labelled a 20% sample of concerns, stratified by arm: 417 concerns from 376 trials, supplying two judgments and a note for every concern. The complete annotations are included with the sample. Labelling was blind to the rule's output and to the model. Each concern received two labels: whether it is about the semantic layer and, when a fault was planted, whether it identifies that fault. Table III compares these labels with the rule; intervals come from a bootstrap over trials (10,000 resamples). For D1, the rule agreed on 94.5% of concerns, $\kappa = 0.89$ [0.85, 0.93] [19]; its precision was 0.995 [0.98, 1.0] and its recall 0.90 [0.86, 0.94]. For D2, on the 219 concerns from fault arms, agreement was 95.4% ($\kappa = 0.90$ [0.84, 0.96]), with precision and recall both 0.96 and five errors in each direction.

TABLE IV
CORRECT AND FOLLOWED RATES BY MODEL (48 TRIALS PER CELL)

	Correct			F1 (coherent)		F2 (contradict.)	
Model	C0	C1	E	Correct	Follow.	Correct	Follow.
Opus	71%	100%	79%	52%	46%	96%	4%
Sol	62%	100%	69%	23%	73%	67%	33%
Haiku	60%	100%	67%	42%	56%	88%	12%

H. Analysis

The four protocol-specified comparisons are paired by question:

- **H1:** following is higher on F1 than on F2;
- **H2:** correctness is higher on E than on F1;
- **H3a:** D2 on F2 is higher with the audit instruction;
- **H3b:** D1 on the correct layer is higher with the audit instruction.

Intervals are 95% cluster-bootstrap intervals over questions (10,000 resamples) [20], with a sensitivity analysis that resamples the six fault operators instead, and a logistic generalized estimating equation (GEE) check [21]. GEE replaces the mixed-effects logistic check named in the draft protocol; the paired bootstrap remains the primary analysis. When an arm has a constant outcome, the logistic check is omitted because of separation. Comparisons added after the pilot are labelled exploratory (X1–X5).

I. Pilot and Amendment

A 180-trial pilot ($4.02) found two benchmark defects, both fixed before the main run. First, the TPC-DS source flag `d_weekend = 'Y'` marks Fridays and Saturdays, so the base layer's weekend definition was itself wrong; question Q22 was redesigned. Second, TPC-DS injects NULLs independently into derived columns, so formulas computed from component columns are now accepted as equivalent. The main run cost $36.41 for 1,512 trials, all of which completed. Total model spend for the study was $40.44.

IV. RESULTS

Table IV gives the main outcomes by model, Fig. 1 shows every arm, and Table V lists all paired comparisons.

A. RQ1: A Correct Layer Helps

Without a layer, agents answered 60–71% of questions correctly; with the correct layer, every model answered every question correctly (Table IV). The pooled difference is $+35$ percentage points [+19, +53] (X4). Without a layer, agents miss business definitions they cannot infer from the schema: the fiscal calendar, which categories count as apparel, the store-size threshold, what "preferred" means. Removing only the anchored element (E) keeps part of the layer's benefit for the rest of the question: E $-$ C0 $= +7$ [+1, +15] (X5). On the 12-question subset, removing metrics (C1a) or descriptions and `ai_context` (C1b) left all 72 responses correct in each arm. The remaining context was sufficient on that subset; these

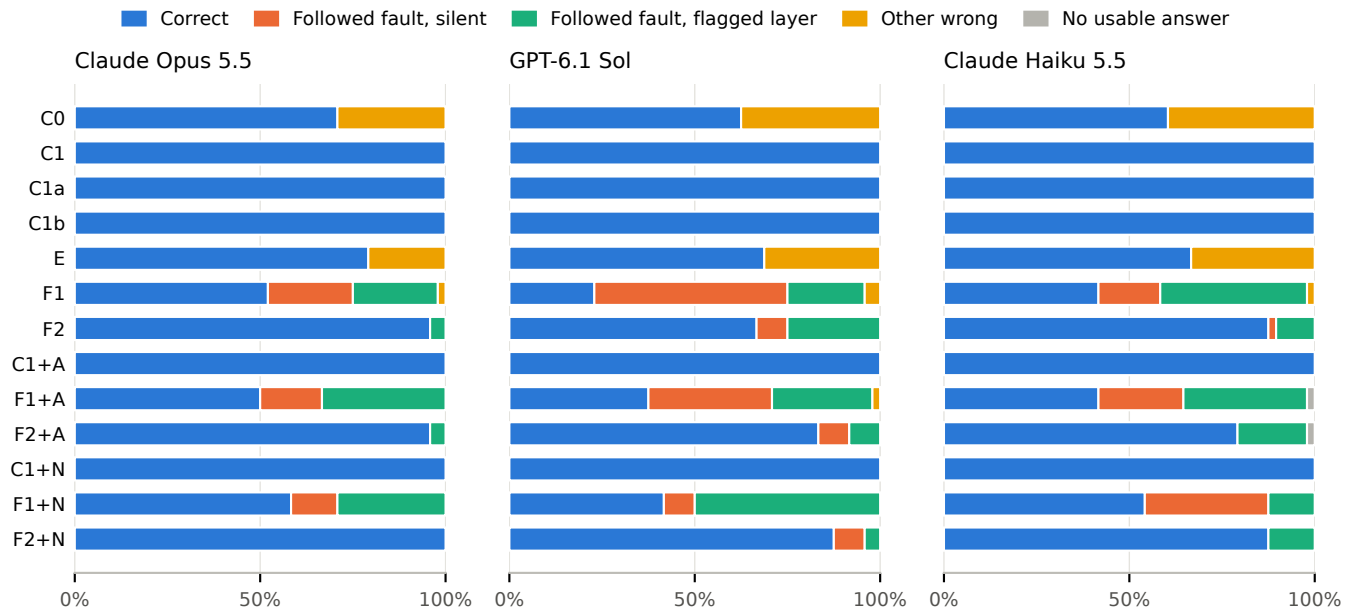


Fig. 1. Outcome of every trial, by model and arm. "Followed fault, flagged layer" means the result equals the faulty definition's result and the same response raised a layer concern (D1). In arms without a fault, a result that happens to equal the faulty result is counted as other wrong. The N, C1a and C1b arms have 24 trials per model; all other arms have 48.

TABLE V
PAIRED COMPARISONS IN PERCENTAGE POINTS, CLUSTERED BY QUESTION. H: PROTOCOL-SPECIFIED; X: EXPLORATORY; RQ2: NAMED IN THE PROTOCOL WITHOUT A SPECIFIED DIRECTION. GEE IS OMITTED FOR X4 BECAUSE C1 IS CONSTANT

ID	Outcome	Contrast	Estimate	95% CI (questions)	95% CI (operators)	GEE p
H1	Followed	F1 − F2	+42	[+27, +56]	[+18, +69]	< 0.001
H2	Correct	E − F1	+33	[+18, +48]	[+11, +54]	< 0.001
H3a	D2	F2+A − F2	−1	[−4, +1]	[−3, +1]	0.346
H3b	D1	C1+A − C1	+17	[+5, +29]	[+4, +29]	0.011
X1	Followed	F2 − F2+A	+3	[−1, +9]	[−3, +10]	0.252
X2	Followed	F1 − F1+A	+3	[−3, +8]	[0, +6]	0.338
X3	Silent	F1 − F1+A	+6	[−3, +15]	[−4, +16]	0.188
X4	Correct	C1 − C0	+35	[+19, +53]	[+8, +67]	–
X5	Correct	E − C0	+7	[+1, +15]	[+1, +16]	0.067
RQ2	Correct	C0 − F1	+26	[+9, +43]	–	–
RQ2	Correct	C0 − F2	−19	[−40, +2]	–	–

ceiling results do not establish which construct carries the gain on other tasks.

B. RQ2: A Wrong Element Is Worse than a Missing One, and Worse than None

With the anchored element removed, accuracy was 67–79%; with a coherent fault in it, 23–52%. H2 is supported: E − F1 = +33 percentage points [+18, +48], robust to operator clusters [+11, +54] and to the GEE check ($p < 0.001$). A coherent wrong element is also worse than no layer at all: C0 − F1 = +26 percentage points [+9, +43], that is +19 for Opus and Haiku and +40 for Sol. This comparison is named in the protocol's RQ2 without a direction specified in advance. We found no clear evidence that contradictory faults reduced accuracy relative to no layer: C0 − F2 = −19 [−40, +2].

C. RQ3: Coherent Faults Are Followed, Sometimes Despite a Located Concern

Coherent faults were followed in 46% (Opus), 73% (Sol) and 56% (Haiku) of trials, and contradictory faults in 4%, 33% and 12%. H1 is supported: F1 − F2 = +42 percentage points [+27, +56], consistent across models (+42, +40, +44 for Opus, Sol and Haiku) and robust to operator clusters [+18, +69] and to the GEE check ($p < 0.001$).

Silent failure, that is following with no layer concern, occurred in 23% of F1 trials for Opus, 52% for Sol and 17% for Haiku (Table VI). A general layer concern (D1) accompanied 19 of Haiku's 27 F1 follows, 11 of Opus's 22 and 10 of Sol's 35. A located concern (D2) accompanied 14/27, 9/22 and 10/35, respectively. The difference matters: a concern about an unrelated metric does not establish that

TABLE VI
DETECTION ON FAULTY AND CORRECT LAYERS. SILENT: SHARE OF F1 TRIALS THAT FOLLOWED THE FAULT WITH NO LAYER CONCERN. LOCATED: F1 FOLLOWS THAT RAISED A D2 CONCERN, OUT OF ALL F1 FOLLOWS

	F1 (coherent)				F2	C1	C1+A
Model	D1	D2	Silent	Located	D2	D1	D1
Opus	77%	73%	23%	9/22	100%	23%	52%
Sol	48%	48%	52%	10/35	92%	2%	6%
Haiku	83%	73%	17%	14/27	98%	35%	52%

TABLE VII
FOLLOWING AND DETECTION BY FAULT OPERATOR, ALL MODELS POOLED (24 TRIALS PER OPERATOR AND ARM)

	F1		F2		F1+A
Operator	Followed	Silent	Followed	D2	Followed
Formula drift	54%	17%	17%	100%	54%
Hidden filter	71%	29%	12%	100%	62%
Synonym	50%	17%	33%	92%	42%
Fiscal definition	100%	67%	0%	100%	100%
Relationship key	0%	0%	0%	100%	0%
Value mapping	75%	54%	38%	88%	75%

the followed fault was noticed. On F2, Sol raised a located concern in 12 of its 16 follows. For example, Opus's Q06 F1 response (repetition 1) described the planted NULL-store filter in `total_quantity` but still applied that filter in its query.

Following depends strongly on the operator (Table VII). No coherent wrong join key was followed, while every coherent fiscal-year fault was followed, two thirds of them silently. In these tasks, schema-visible conflicts coincided with lower following: joining `ss_addr_sk` to `c_customer_sk` conflicts with the column names' apparent meanings. Following was higher for plausible business conventions: a July fiscal start, a 240-employee threshold, which categories count as apparel.

D. RQ4: An Audit Instruction Raises Concerns Without a Clear Reduction in Following

H3a is not supported: located detection on F2 is already at ceiling (92–100%), and the audit instruction changes it by -1 point [-4, +1]. H3b is supported: the audit raises concerns on the correct layer by $+17$ percentage points [+5, +29] (operator clusters [+4, +29]; GEE $p = 0.011$), from 23% to 52% for Opus, 2% to 6% for Sol and 35% to 52% for Haiku.

In exploratory comparisons, the audit barely reduced following: by 3 percentage points [-1, +9] on contradictory faults (X1), where Sol gained most ($+17$) and Haiku did slightly worse (-6), and by 3 percentage points [-3, +8] on coherent faults (X2). Naming the suspect element (F1+N) raised located detection of coherent faults to 88% for Opus, 92% for Sol and 62% for Haiku, yet agents still followed the fault in 42–58% of those trials.

Concerns on the reference layer are not all false alarms. On C1, they frequently targeted `total_profit`: its description uses component arithmetic while its expression uses a derived column. Other concerns addressed definitional choices in customer lifetime value and average ticket value, or the example's "manufacturer" synonym for `i_brand`. The author labels establish whether a concern is about the layer, rather than whether it is justified; separating substantive concerns from noise remains open.

V. DISCUSSION

Coherent faults received less resistance. Models located documentation and schema conflicts more often than coherent departures from the reference. A plausible policy change may leave no visible conflict. Such cases motivate independent policy references rather than assuming the model can recover an undisclosed convention.

Concerns require a response policy. Some responses raise a located concern and still use the associated definition. A pipeline that displays the number but hides the concern can therefore conceal a detected problem. Products should surface relevant concerns or consider abstaining when a concern names an element used by the query.

Prompting for skepticism. The generic audit raised concerns without a clear reduction in following. Verifying a coherent convention can require an approved policy, ledger reconciliation or independent tests; data profiles alone may not establish business intent. Follow-up work could test tool-using agents and semantic layers carrying assertions, such as reconciliation of a sales metric with a finance ledger.

Model differences. GPT-6.1 Sol had the highest observed following and silent-following rates. Claude Opus 5.5 had the lowest following rate, while Claude Haiku 5.5 often raised concerns alongside followed answers. These comparisons describe the tested prompts and tasks rather than general model traits.

Implications for Ossie. Building the benchmark surfaced two problems in the official example: a metric whose plain-SQL reading double-counts under joins, and a misleading synonym ("manufacturer" for brand). A linter that checks descriptions against expressions, and flags model-scoped metrics with ambiguous join paths, could help identify such inconsistencies; we did not implement or evaluate that linter.

VI. THREATS TO VALIDITY

Scale. There are 24 questions on one public schema, and models may know TPC-DS conventions. The intervals are wide and question-level, and the 12-question subset arms are smaller still.

Injected faults. The faults are injected, not natural, and their plausibility is argued rather than measured. The author wrote the layer additions, the faults and the first gold set; the second gold set was written by a model. The Q22 redesign and the equivalence grading changed after the pilot and before the main run, and both are documented.

Base-layer imperfections. Some descriptions in our base layer do not match their expressions exactly (for example

`total_profit`), which raises concerns on the correct layer. All base-layer edits are logged.

Detection is rule-based and conservative. Against the author's labels, the rule almost never over-counts D1 (precision 0.995). It misses about one layer concern in ten, and two kinds account for almost all the misses: concerns that the layer *lacks* a definition, which name no layer identifier, and layer concerns filed under the question or the data. Misses concentrate in arm E, where D1 agreement is 78%, so D1 rates can be underestimated, especially for E, and missed concerns can inflate measured silent failure. D2 errors are balanced. Free-text concerns that do not use the structured `object` field are not counted.

Limits of the human check. It used one rater. The original sample display described only `fiscal_year` for Q13–Q16, although the fault also changed `fiscal_quarter`. The author notes explicitly consider both fields. As a sensitivity analysis, excluding these 34 concerns leaves D2 agreement unchanged at $\kappa = 0.90$ (Table III).

Single turn, no tools. The results do not describe agents that can query the data.

Literature. The related-work comparison is targeted rather than systematic, so relevant studies may be missing.

Statistics. The GEE check is omitted for X4 because every model scores 100% on C1. Operator-cluster intervals rely on only six clusters and should be interpreted cautiously.

VII. Conclusion

On this benchmark, a correct semantic layer raised accuracy from 60–71% to 100%, while coherent faults were followed in 58% of trials and left agents worse off than no layer. Some responses used a definition despite a located concern, and a generic audit instruction mostly added concerns. Independent policy evidence, relevant-concern handling and checkable assertions are promising directions for semantic-layer reliability.

Data and Code Availability

The repository at https://github.com/barveP/semantic-layer-trust-paper contains benchmark inputs, prompts, frozen plans, graded trials, analysis code and all 417 author annotations with notes. The documented build regenerates the result and audit tables. Historical freeze records and later reporting changes are distinguished in the README. Full experiment reruns additionally require Snowflake, generated TPC-DS data and an external dispatcher not distributed here. Credentials and unparsed responses are excluded.

Acknowledgment

OpenAI Codex assisted with manuscript revisions and analysis-code changes. All human-audit judgments and notes were supplied by the author.

References

[1] Apache Ossie contributors, "Apache Ossie (incubating)," https://github.com/apache/ossie, 2026, revision 2fd19c7a, accessed 31 Oct. 2026.

[2] M. Rumiantsau and I. Fokeev, "Semantic layers for reliable LLM-powered data analytics: A paired benchmark of accuracy and hallucination across three frontier models," https://arxiv.org/abs/2604.25149, 2026.

[3] H. J. Kim, S. Khoeurn, and Y. J. Yoon, "A semantic-layer-mediated agent for natural language to SQL over heterogeneous enterprise databases," https://arxiv.org/abs/2606.31041, 2026.

[4] A. S. Pillai, "GROUND: Reducing hallucinations in LLM-based enterprise analytics through governed semantic definitions," https://arxiv.org/abs/2608.26157, 2026.

[5] M. Lee, "Business truth, not SQL accuracy: A rule-gated 7B analytics agent outperforms a direct-prompted 32B baseline," https://arxiv.org/abs/2608.09254, 2026.

[6] Transaction Processing Performance Council, "TPC benchmark DS," https://www.tpc.org/tpcds/.

[7] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] J. Yun and S. goo Lee, "SEED: Enhancing text-to-SQL performance and practical usability through automatic evidence generation," in *Proc. IEEE Int. Conf. Data Engineering Workshops (ICDEW)*, 2025, arXiv:2506.07423.

[9] T. Jin, Y. Choi, Y. Zhu, and D. Kang, "Pervasive annotation errors break text-to-SQL benchmarks and leaderboards," https://arxiv.org/abs/2601.08778, 2026.

[10] S. Chang, J. Wang, M. Dong, L. Pan, H. Zhu, A. H. Li, W. Lan, S. Zhang, J. Jiang, J. Lilien, S. Ash, W. Y. Wang, Z. Wang, V. Castelli, P. Ng, and B. Xiang, "Dr.Spider: A diagnostic evaluation benchmark towards text-to-SQL robustness," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[11] T. Zhang, K. Qian, S. Sahai, Y. Tian, S. Garg, H. Sun, and Y. Li, "EvoSchema: Towards text-to-SQL robustness against schema evolution," https://arxiv.org/abs/2603.10697, 2026.

[12] H. Wang, Y. Chi, J. Yang, L. Yang, J. Zhao, Y. Chen, and G. Chen, "Beyond static rules: Automated discovery of latent vulnerabilities in text-to-SQL," https://arxiv.org/abs/2607.03833, 2026.

[13] L. Tu, R. Wang, T. Sheng, S. Ravi, and D. Roth, "LLM NL2SQL robustness: Surface noise vs. linguistic variation in traditional and agentic settings," https://arxiv.org/abs/2603.17017, 2026.

[14] M. Chen and S. Arifuzzaman, "Dual-axis benchmark and preventive scaffold: Surfacing and mitigating silent risks in LLM data preparation," https://arifuzzaman.faculty.unlv.edu/paper/icmla26-llm.pdf, 2026.

[15] R. Xu, Z. Qi, Z. Guo, C. Wang, H. Wang, Y. Zhang, and W. Xu, "Knowledge conflicts for LLMs: A survey," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[16] W. Macke and M. Doyle, "Testing the effect of code documentation on large language model code understanding," in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024, pp. 1044–1050.

[17] M. Raasveldt and H. Mühleisen, "DuckDB: an embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019.

[18] T. Mao, "SQLGlot," https://github.com/tobymao/sqlglot, version 25.34.1.

[19] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[20] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B*, vol. 69, no. 3, pp. 369–390, 2007.

[21] K.-Y. Liang and S. L. Zeger, "Longitudinal data analysis using generalized linear models," *Biometrika*, vol. 73, no. 1, pp. 13–22, 1986.